

MINI · QUICKSTART · v3

7

ก้าว

Desk-Pet บน ESP32

ไม่ต้องแตะ ESP-IDF · ไม่ต้องมีบอร์ดก็เริ่มได้



วาด GIF

character pack

LittleFS

web flasher

capture

ChaiKlang Oracle (ชายกลาง) · **The Middle Switchboard** 🎮

คำนำ

เล่นยาวเล่าทั้ง การเดินทาง — เล่นนี้เป็น มิติ: 7 ก้าวลงมือจริง พา ChaiKlang ขึ้นจอ ESP32 เป็น desk-pet โดย **ไม่ต้องลง ESP-IDF, ไม่ต้องมีบอร์ดก็เริ่มได้** เปิดมาก็ทำตามได้เลย ทุกคำสั่งของจริง 🐾

ก้าวที่ 1 — วาด pet GIF เอง

pet 1 ตัว = โพลเดอร์ GIF 96×100 (GIF89a) 7 สเตท: `sleep idle busy attention celebrate dizzy`
`heart` วาดเองด้วย Pillow → original 100% (MIT สะอาด ไม่ติดลิขสิทธิ์)

```
# กฎเหล็กให้ decode บนชิปไม่พัง: ทุกเฟรม full-size, ไม่มี local palette, disposal เรียบ  
frames[0].save("idle.gif", save_all=True, append_images=frames[1:],  
               duration=160, loop=0, disposal=2, optimize=False)
```

กับดัก: ถ้าเฟรมเป็น partial/offset หรือมี palette ต่อเฟรม → AnimatedGIF บนชิป "พังกลาง animation"
(แต่ browser หนได้) เช็คเร็ว ๆ: ทุกเฟรมต้อง size = (96,100)

ก้าวที่ 2 — ทำ character pack

วาง GIF ในโฟลเดอร์ชื่อ pack + `manifest.json` (schema เดียวกับ clawd):

```
{ "name": "chaiklang", "colors": { "body": "#F0B429", "bg": "#1A1710" },  
  "states": { "sleep": "sleep.gif", "idle": ["idle.gif"], "busy": "busy.gif",  
              "attention": "attention.gif", "celebrate": "celebrate.gif",  
              "dizzy": "dizzy.gif", "heart": "heart.gif" } }
```

นี่คือ "artifact ที่พกพาได้" — ตัว firmware ไม่ต้องสร้างเอง

ก้าวที่ 3 — "many bodies, one soul" (เช็ค GIF ใน browser)

GIF ชุดเดียวกันนี้ decode ได้ทั้งบน **device** (AnimatedGIF native) และ **browser** (gif-wasm/emcc) — decoder ตัวเดียว หลายร่าง ใช้ browser พรีวิวก่อนได้:

```
python3 -m http.server 8011 -d gif-wasm/web # เปิดดู pack ใน Canvas — ถ้าเล่นในนี้ได้ บนชิพ
```

ถ้าเล่นใน gif-wasm ครบ 7 สเตท = มั่นใจว่า decode บนชิพผ่าน (engine เดียวกัน)

ก้าวที่ 4 — build LittleFS storage เอง (ไม่ต้อง ESP-IDF)

กุญแจ: pet app **discover pack** แรกใน LittleFS เอง (`find_first_pack`) → ไม่ต้อง build firmware ทำแค่ **storage image** ด้วย Python:

```
# pip/uv - ไม่มี toolchain ก็ได้
from littlefs import LittleFS
fs = LittleFS(block_size=4096, block_count=0x300000//4096) # 3MB partition
fs.makedirs("/characters/chaiklang", exist_ok=True)
# ...เขียน gif ทุกไฟล์ + manifest.json ลงไป...
open("chaiklang-storage.bin", "wb").write(bytes(fs.context.buffer))
```

ก้าวที่ 5 — flasher manifest (reuse shared app)

ใช้ bootloader/partition/app ที่ build ไว้แล้ว + storage ของเรา:

```
"parts": [
  {"path": "bootloader.bin", "offset": 0},           // ต้อง 0xE9 (CI flasher-check)
  {"path": "partition-table.bin", "offset": 32768},
  {"path": "jc3248_pet_idf-clawd.bin", "offset": 65536}, // shared app, discover pack
  {"path": "chaiklang-storage.bin", "offset": 2686976} ] // 0x290000 = LittleFS ของเรา
```

กับดัก: byte 0 ของ part offset-0 ต้องเป็น **0xE9** (ESP image) — ถ้าเป็น 0xff = brick · เช็ค `xxd -l2`

`bootloader.bin`

ก้าวที่ 6 — ลงทะเบียนใน web flasher

flasher เป็น data-driven — หย่อน `docs/packs/<id>.json` ไฟล์เดียว ไม่ต้องแก้ index.html (ไม่ชน merge):

```
{ "id":"chaiklang","name":"ChaiKlang 🐱","kind":"pet",  
  "manifest":"manifest-chaiklang.json","preview":"preview/gifs/chaiklang/idle.gif" }
```

`kind:"pet"` → เข้า picker + live preview (animate) · `"firmware"` → การ์ดเฉย ๆ ไม่มี preview

ก้าวที่ 7 — capture & verify (เห็นด้วยตา)

ไม่มีบอร์ดก็พิสูจน์ได้: render LVGL เดิมบน host+SDL, capture หน้าต่างเดียวบน macOS:

```
screencapture -o -x -l <WINDOW_ID> f.png          # window-only (Quartz หา id)
ffmpeg -framerate 8 -i 'f%03d.png' -c:v libx264 out.mp4
# wasm canvas ใน headless Chrome ไม่ขึ้นจอดำ:
chrome --headless=new --enable-unsafe-swiftshader --use-angle=swiftshader --virtual-
```

ปิดท้าย: วาด → pack → LittleFS → flasher → capture · ไม่แตะ ESP-IDF เลย · GIF ชัดเดียวเล่นทั้ง browser และชิป — หลายร่าง วิญญาณเดียว 🦊🐼